

Redis :

For a senior/architect interview, cache questions are usually not about **"What is Redis?"** They are about what happens when caching goes wrong: stale data, cache stampede, hot keys, eviction, memory pressure, consistency, failures, penetration, avalanches, distributed locking, invalidation, and what you do when Redis itself becomes unavailable.

1. Cache-Aside Pattern

In one of my backend systems, we used Redis as a cache in front of PostgreSQL because some product and configuration data was read much more frequently than it changed. We followed the cache-aside pattern. Whenever a request came in, the service first checked Redis. If the key existed, we returned the cached value immediately. If Redis returned a miss, the service queried PostgreSQL, returned the result to the client, and populated Redis with a TTL. This reduced database load and improved response latency significantly. The important point was that Redis was not treated as the source of truth; PostgreSQL remained authoritative, and Redis was considered a performance optimization.

Request

|

↓

Redis

|

|— HIT → Return

|

|— MISS

↓

Database

↓

Redis

↓

Return

2. Cache Invalidation Problem

One of the most common issues we faced was **stale cache data**. Suppose a product price was ₹1,000 in Redis and PostgreSQL, and an administrator changed the price to ₹900 in PostgreSQL. If Redis wasn't invalidated, users could continue receiving ₹1,000. We solved this by treating the database as the source of truth and invalidating or updating the cache whenever the underlying data changed. For critical updates, the write flow became database update → cache invalidation, and for distributed systems we also used events so that other services could invalidate their local caches. We also used TTL as a safety mechanism, but we didn't depend on TTL alone because that could leave stale data for the entire TTL duration.

Write

|

↓

Database

|

↓

Invalidate Redis

A strong interview line is:

"TTL limits the maximum staleness window, but TTL alone is not a complete cache invalidation strategy."

3. Cache-Aside vs Write-Through

In some use cases, cache-aside was sufficient, but for data that required a higher cache hit ratio we considered write-through caching. With cache-aside, the application writes to the database and separately manages the cache. With write-through, the application writes through the cache layer and the cache updates the underlying persistent store. The choice depended on the access pattern and consistency requirements. I generally prefer cache-aside for application-controlled caching because it keeps the database as the authoritative source and is easier to reason about, while write-through can be useful when cache freshness and write coordination are important.

4. Cache Stampede / Thundering Herd

One serious problem we considered was a **cache stampede**. Imagine a very popular product cached by Redis with a TTL of five minutes. When the key expired, thousands of requests arrived almost simultaneously. All of them saw a cache miss and all of them queried PostgreSQL at the same time. The database suddenly received thousands of identical queries even though normally Redis had protected it. We solved this using request coalescing or a distributed lock so that only one request fetched the data from the database while the others waited for the cache to be populated. We also used TTL jitter so that many keys didn't expire at exactly the same time.

Before:

1000 requests

|

↓

Redis MISS

|

+-----> DB

+-----> DB

+-----> DB

+-----> DB

+-----> DB

After:

1000 requests

|

↓

Redis MISS

|

↓

Lock

|

↓

One request → DB

|

↓

Redis populated

|

↓

Other requests → Redis

5. Cache Avalanche

Another problem is a **cache avalanche**, where a large number of cache entries expire around the same time. For example, if we loaded thousands of keys at deployment time with exactly the same one-hour TTL, they could all expire at 2 PM. At that point, a huge number of requests would miss the cache simultaneously and hit the database. We mitigated this by adding randomized TTL jitter, warming important keys, avoiding synchronized expiration, and using multiple layers of caching where appropriate. For example, instead of every key having exactly 3600 seconds, we could use a TTL around $3600 \pm \text{random interval}$.

Bad:

Key A → 3600 sec

Key B → 3600 sec

Key C → 3600 sec

Key D → 3600 sec

↓

All expire together

↓

Huge DB load

6. Hot Key Problem

We also need to handle **hot keys**, where one cache key receives a disproportionately large percentage of traffic. For example, if a celebrity product, popular event, or common configuration key receives 50,000 requests per second, even though Redis can handle large traffic, one logical key can become a bottleneck depending on the architecture. We can solve this using local in-process caching, key replication or sharding techniques where appropriate, CDN caching for suitable data, and request coalescing. The key point is to identify hot keys through metrics instead of assuming that total Redis throughput alone represents the problem.

50K requests

|

↓

product:123

|

↓

One extremely hot key

Possible solution:

L1 Cache → Application memory

↓

L2 Cache → Redis

↓

Database

7. Cache Penetration

Another issue is **cache penetration**, where requests repeatedly ask for data that doesn't exist. For example, an attacker could send thousands of requests for random product IDs such as `product:999999999`. Redis doesn't contain those keys, PostgreSQL doesn't contain them either, and every request therefore reaches the database. We handled this

by caching negative results for a short TTL, validating identifiers where possible, applying rate limiting, and in some high-risk scenarios using Bloom filters to determine whether a key could exist before hitting the database.

product:999999

|

↓

Redis MISS

|

↓

DB MISS

|

↓

Repeat forever

Solution:

DB says NOT FOUND

↓

Cache "NOT_FOUND"

↓

Short TTL

8. Cache Penetration vs Stampede vs Avalanche

This distinction is very useful in interviews.

Cache penetration means requests are asking for data that doesn't exist, so every request bypasses the cache.

Cache stampede means one or a small number of popular keys expire and many concurrent requests simultaneously go to the database.

Cache avalanche means a large number of cached keys expire around the same time, causing a large load spike against the underlying system.

A concise way to remember it is:

Penetration → key doesn't exist

Stampede → popular key expires

Avalanche → many keys expire together

9. Cache Inconsistency

Another challenge is **cache inconsistency**, especially in distributed systems. Suppose Service A updates PostgreSQL and Service B maintains a Redis cache containing the same data. If Service A successfully updates PostgreSQL but the cache invalidation message fails, Service B can continue serving old information. We solved this by using reliable event propagation, transactional outbox patterns where appropriate, idempotent consumers, and TTL as a final safety mechanism. The important architectural decision was that Redis remained a derived representation rather than the authoritative source.

PostgreSQL

|

↓

Outbox

|

↓

Kafka

|

↓

Cache invalidation

This is more reliable than simply doing:

DB update

↓

Kafka publish

because a failure between the two operations can otherwise leave the system inconsistent.

10. Cache Invalidation vs Cache Eviction

These two concepts are frequently confused. **Invalidation** means we intentionally say that a cached value is no longer valid, usually because the underlying data changed. **Eviction** means the cache removes an entry because of a memory limit or an eviction policy such as LRU. For example, if a product price changes, that's an invalidation event. If Redis runs out of memory and removes an old key according to its eviction policy, that's eviction. I treat them as separate mechanisms: invalidation maintains correctness, while eviction manages cache capacity.

Invalidation:

Data changed → remove/update cache

Eviction:

Memory pressure → cache removes key

11. Redis Outage

A critical production scenario is when Redis itself becomes unavailable. We don't want a cache failure to become a complete application failure because the cache is an optimization layer. Our application should have a graceful fallback path where a Redis timeout or connection failure causes the service to bypass the cache and query the database, subject to protection mechanisms. We also use short Redis timeouts, connection-pool limits and circuit breakers so that thousands of application requests don't wait indefinitely for Redis.

Request

|

↓

Redis

X

|

Circuit breaker

|

↓

Database

But there is an important caveat: if Redis normally handles 95% of traffic and suddenly all traffic goes to PostgreSQL, the database may itself become overloaded. Therefore cache fallback must be combined with **rate limiting, database protection, load shedding and sometimes degraded responses**.

12. Cache Failure Causing Database Failure

This is one of the strongest architect-level scenarios.

Suppose our application normally receives 100,000 requests per second, Redis handles 90,000 of them, and PostgreSQL receives only 10,000. If Redis fails, suddenly PostgreSQL can receive close to 100,000 requests per second. PostgreSQL becomes saturated, latency increases, requests timeout, clients retry, and the retry traffic makes the situation even worse. Therefore I would never design cache fallback as simply "Redis fails, query DB." I would put a bounded fallback path around the database with rate limits, connection pools, circuit breakers, request prioritization and possibly stale/local cached data.

Normal:

100K requests

|

Redis

|

90K hits

|

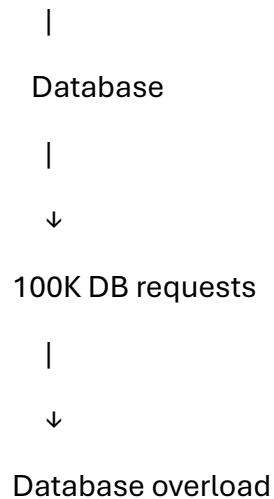
10K DB

Redis failure:

100K requests

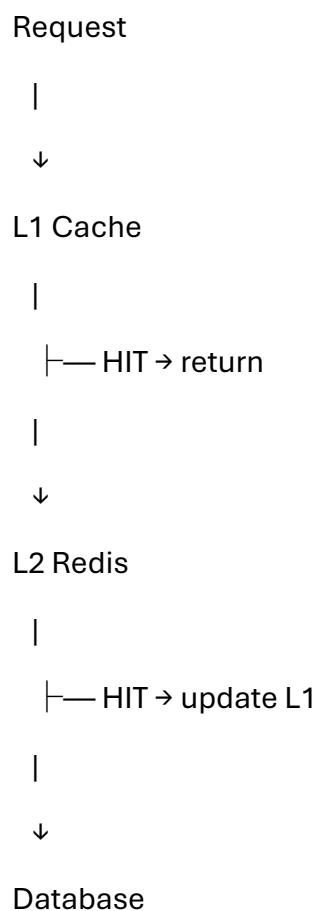
|

Redis X



13. Local Cache + Distributed Cache

For extremely hot read paths, we can use two cache layers. The first is an in-process L1 cache and the second is a distributed L2 cache such as Redis. A request first checks local memory because it is extremely fast. If the local cache misses, it checks Redis, and only then does it access the database. The trade-off is that local caches introduce additional consistency complexity because every application instance has its own copy.



14. Local Cache Inconsistency

With L1 caching, suppose we have:

Pod A → product price = 100

Pod B → product price = 100

The price changes:

Database → 90

Redis → 90

But Pod A's local cache may still contain:

Pod A → 100

Therefore local caching needs an invalidation mechanism or a short TTL. In a distributed architecture, we can publish invalidation events through Kafka or another messaging system so that every application instance can invalidate its local entry.

DB

↓

Event

↓

Kafka

↓

Pod A → invalidate

Pod B → invalidate

Pod C → invalidate

15. Cache Consistency with Concurrent Writes

Suppose two requests update the same product simultaneously:

Request A → price = 100

Request B → price = 120

If both update the database and cache independently, ordering problems can result in the cache containing an older value. We can address this using database versioning or

timestamps and only allowing a cache update when the incoming version is newer than the cached version. Optimistic concurrency control can also protect the database from lost updates.

For example:

Product version = 5

Request A:

version 5 → 6

Request B tries:

WHERE version = 5

and fails because the database is already at version 6.

16. Cache Race Condition

A common race is:

Request A:

DB update → 200

|

↓

Cache update

Request B:

Cache update

Depending on timing, an older value may overwrite a newer value.

For critical data, we can use:

- Version numbers
- Optimistic locking
- Atomic Redis operations
- Distributed coordination where necessary
- Event ordering

The key principle is:

Don't assume cache writes are automatically ordered just because database writes are transactional.

17. Cache TTL Problem

TTL is useful but selecting the wrong TTL creates problems. If TTL is too long, stale data can remain available for too long. If TTL is too short, the cache hit ratio drops and the database receives more traffic. I choose TTL based on how frequently the data changes and how much staleness the business can tolerate. For highly dynamic data such as inventory, I would use a much shorter TTL or explicit invalidation, while relatively static configuration or product metadata can have longer TTLs.

TTL too long

↓

stale data

TTL too short

↓

low hit ratio

↓

high DB load

18. Cache Warming

After deployment or restart, the cache can be empty. If millions of requests arrive immediately, the application may generate a large number of cache misses. We can solve this with cache warming, where we preload the most frequently accessed keys before or during traffic ramp-up. For example, after deployment we might preload popular product data or configuration into Redis. However, I would only warm high-value keys because attempting to preload the entire database can itself create unnecessary load.

Deployment

↓

Warm popular keys

↓

Receive traffic

↓

High cache hit ratio

19. Cache Eviction

A cache has finite memory. When memory becomes full, it needs an eviction policy.

Common policies include:

LRU

LFU

TTL-based expiration

Random eviction

LRU

Least Recently Used

Remove data that hasn't been accessed recently.

LFU

Least Frequently Used

Remove data that has been accessed less frequently.

The correct policy depends on the workload. For example, if a small number of keys are extremely popular, LFU can be useful because frequency is important. The key point is that eviction is about managing finite memory, not maintaining business correctness.

20. Redis Memory Exhaustion

A production Redis cluster can experience memory pressure because of:

- Too many keys
- Large values
- Missing TTLs
- Incorrect data structures
- Large lists/sets
- Unexpected traffic

- Memory fragmentation

We monitor:

used_memory

memory fragmentation

evictions

hit ratio

key count

command latency

connections

CPU

We also set appropriate maxmemory and an appropriate eviction policy rather than allowing Redis to consume all available machine memory.

21. Cache Serialization Problem

Another issue is serialization overhead. Suppose we store huge JSON objects in Redis:

```
{  
  "product": "...",  
  "reviews": [...],  
  "recommendations": [...],  
  "metadata": [...]  
}
```

Every request requires serialization/deserialization and transfers a large payload. We can improve this by caching only the data needed by the hot path, using efficient serialization where appropriate, and avoiding unnecessarily large values.

A useful interview statement is:

"Caching everything is not necessarily good caching; I optimize for access pattern, object size, hit ratio and invalidation complexity."

22. Cache Key Design

Bad key design can create collisions or make invalidation difficult.

Instead of:

123

use a structured key:

product:v1:123

For tenant-aware systems:

tenant:456:product:v1:123

This prevents one tenant's cached data from accidentally being returned to another tenant.

For APIs:

user:v1:123

product:v2:456

search:v1:electronics:page:1

Versioning keys can also make schema changes safer.

23. Multi-Tenant Cache Problem

In a multi-tenant architecture, one major security problem is accidentally sharing cache entries between tenants.

Suppose:

Tenant A

user_id = 123

and:

Tenant B

user_id = 123

If the key is:

user:123

both tenants could resolve to the same cache entry.

Instead:

tenant:A:user:123

tenant:B:user:123

This is both a correctness and security requirement.

24. Cache Security

Sensitive information should not automatically be cached. We need to consider:

- Authentication tokens
- Password-related data
- Personal information
- Financial information
- Authorization data

If sensitive data must be cached, we need appropriate access controls, encryption/security controls, TTLs and isolation.

Also, never assume that because Redis is "internal," sensitive data is automatically safe.

25. Cache and Distributed Lock

Sometimes only one process should perform an expensive operation.

For example:

1000 requests

|

↓

Cache MISS

Without coordination:

1000 → DB

With a lock:

Request 1 → acquire lock → DB

Request 2 → wait

Request 3 → wait

...

Once Request 1 populates the cache:

Redis populated

↓

Others read cache

But distributed locks should be used carefully. They introduce their own failure and timeout problems, and shouldn't be used merely because "Redis has locks."

26. Cache Failure During a Traffic Spike

Suppose normal traffic is:

50K RPS

Redis handles:

45K

and DB handles:

5K

Suddenly Redis becomes unavailable during a traffic spike.

If we simply send all requests to PostgreSQL:

50K → PostgreSQL

the DB may fail.

My approach would be:

Redis failure

↓

Circuit breaker

↓

Rate limit fallback traffic

↓

Serve stale/local data where safe

↓

Prioritize critical operations

↓

Reject/degrade non-critical requests

For example:

Payment → preserve

Order status → preserve

Recommendations → disable

Analytics → delay

This is called **graceful degradation**.

27. Stale-While-Revalidate

For data where slight staleness is acceptable, we can use stale-while-revalidate.

Suppose:

Cache expired

Instead of forcing every request to wait for a fresh DB query:

Request

↓

Stale cached data

↓

Return immediately

Background:

↓

Refresh cache

This is useful for:

- Product catalog
- Configuration
- Recommendations
- Public content

but not necessarily for authoritative financial balances or inventory.

28. Cache and Database Write Ordering

Suppose we do:

Cache update

↓

Database update

and the database update fails.

Now cache contains data that isn't actually persisted.

That is dangerous for authoritative data.

Another approach:

Database update

↓

Cache invalidation

This is often easier to reason about with cache-aside.

The database remains authoritative.

29. Cache Delete vs Cache Update

Suppose:

DB:

price = 100

Redis:

price = 100

Price changes to 90.

Two approaches exist.

Update cache

DB → 90

Redis → 90

Invalidate cache

DB → 90

Redis → DELETE

The next read repopulates Redis.

For many systems, invalidation is safer because it avoids accidentally putting a partially computed or incorrectly transformed value into the cache.

30. Cache Invalidation Race

Consider:

T1:

Update DB → 90

T2:

Read DB → 100

Then:

T1:

Invalidate cache

T2:

writes old 100 into cache

Now cache contains stale data even though the database is correct.

This is a subtle distributed-system race.

Solutions can include:

- Versioned cache values
- Write-through patterns
- Ordered events
- Delayed double deletion in some specific cache-aside designs

- Optimistic concurrency
- Stronger synchronization where business requirements justify it

The important interview point is:

Cache invalidation is a distributed consistency problem, not merely a DELETE command.

31. Cache Observability

I would monitor cache behavior with metrics such as:

Cache hit ratio

Cache miss ratio

Eviction count

Memory usage

Key count

Command latency

Connection count

CPU

Network traffic

Hot keys

TTL distribution

Error rate

A very important metric is:

Hit Ratio = Cache Hits / Total Cache Requests

For example:

90,000 hits

10,000 misses

Hit ratio = 90%

But don't optimize blindly for hit ratio. A 99% hit ratio isn't necessarily useful if the 1% misses are extremely expensive or if the cached data is incorrect.

32. Cache vs Database Consistency Story

An interviewer may ask:

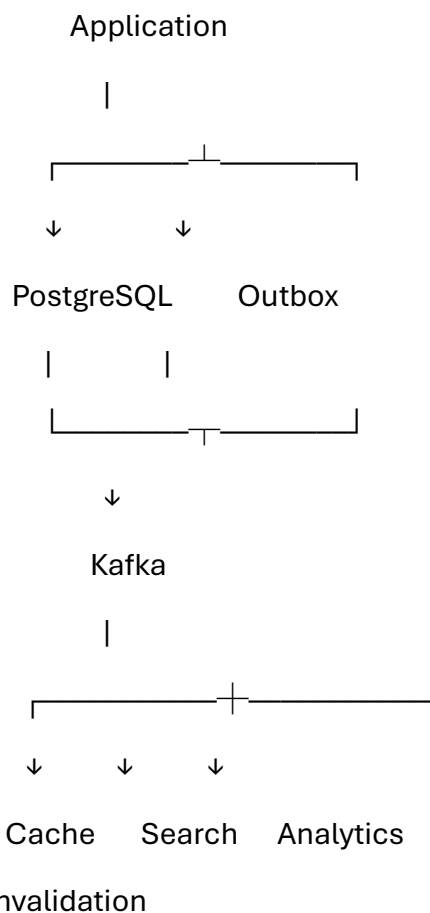
"What happens if cache and DB have different data?"

A strong answer:

"I treat the database as the source of truth and Redis as a derived performance layer. If the cache is stale, I invalidate or refresh it rather than treating the cache as authoritative. For distributed invalidation, I use reliable event propagation such as an outbox and Kafka when necessary. TTL provides a bounded staleness safety net, while versioning or optimistic concurrency can prevent an older value from overwriting a newer cached value."

33. Cache vs Event-Driven Invalidation

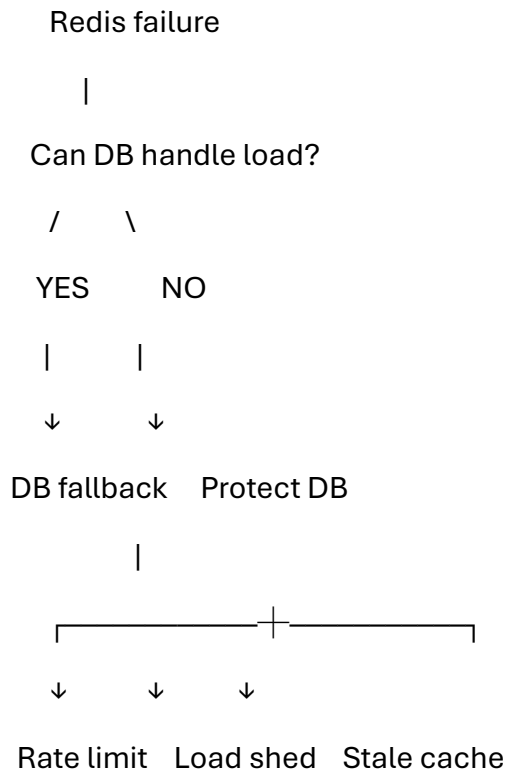
A production architecture can look like:



This gives you a reliable way to propagate changes.

34. Cache Failure Decision Tree

When Redis fails:



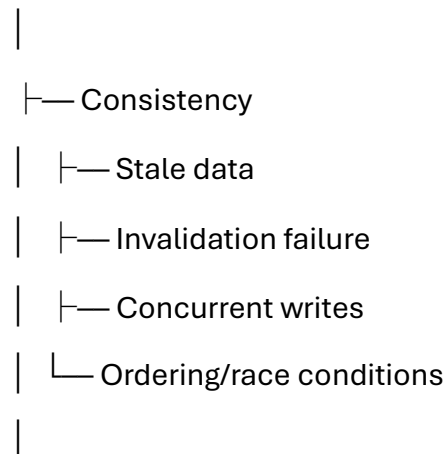
This is much better than simply saying:

"If Redis fails, use PostgreSQL."

35. The complete cache problem map

For interviews, remember these categories:

CACHE PROBLEMS



└─ Load Problems

| └─ Cache stampede

| └─ Cache avalanche

| └─ Hot keys

| └─ Cache penetration

|

└─ Capacity

| └─ Memory exhaustion

| └─ Eviction

| └─ Large values

| └─ Too many keys

|

└─ Availability

| └─ Redis outage

| └─ Network failure

| └─ Connection exhaustion

| └─ Failover

|

└─ Performance

| └─ Low hit ratio

| └─ Serialization overhead

| └─ Network latency

| └─ Hot partitions

|

└─ Security

| └─ Tenant isolation

| └─ Sensitive data

- | └─ Authorization leakage
- |
- └─ Operations
 - └─ Cache warming
 - └─ Monitoring
 - └─ TTL tuning
- └─ Capacity planning

36. The interview story I'd recommend memorizing

"In a high-traffic service, we introduced Redis as a cache in front of PostgreSQL using the cache-aside pattern. The database remained the source of truth. Initially, we encountered stale data because database updates weren't immediately reflected in Redis, so we introduced explicit invalidation and TTL as a safety mechanism. We then noticed that when popular keys expired, many concurrent requests hit PostgreSQL simultaneously, creating a cache stampede. We addressed that using request coalescing/distributed locking for selected hot keys and randomized TTLs. We also handled cache penetration by caching short-lived negative results and validating requests. For Redis failures, we used short timeouts and circuit breaking, but we didn't blindly send all traffic to PostgreSQL because that could overload the database; instead we used rate limiting, bounded DB connection pools, load shedding and stale/local data where the business allowed it. We monitored hit ratio, miss ratio, eviction rate, memory utilization, latency and hot keys. For distributed invalidation, we used reliable event propagation, such as an outbox with Kafka, so that cache invalidation wasn't lost between a successful database transaction and event publishing."

That paragraph demonstrates **far more senior-level cache knowledge** than simply saying *"we used Redis to improve performance."*